

# Guardrails

**A hands-on course in building agents by building their harness**

Lesson 6

# The box limits reach, not whether

Inside Lesson 5's box, the agent still runs whatever the model asks, as soon as it asks, for as long as it keeps asking.

- `rm -rf` on the mounted folder: your real project, gone
- Stuck retrying one command: it goes on until you notice the bill

# The model proposes; the harness decides

Guardrails are rules about what the harness is allowed to do, checked before it does it: which commands may run, when a person gets a say, when a run has gone on long enough.

**Built on control flow**, which decides whether the next step happens at all:

- It turns a tool request into a command being run
- It turns the result into another model call
- A guardrail is a decision at one of those two points, so it lives there

## A few checks, where the loop already passes

- **A gate before each tool:** *allow* it, *ask* a person first, or *deny* it
- **A refusal the model can read:** the refusal goes in the result's slot, marked as an error, saying why
- **Limits before each call:** count the steps and tokens; past the limit, stop
- **An interrupt:** press ESC and quark stops, keeps what it had done, tells the model, and hands the run back to you

## The box comes first

- Guardrails don't change what the model asks for, only whether the harness goes along with it
- They're only as good as the rules: a check is code reading a command
- A command can say the same thing many ways
- The guard keeps out what you'd never want; the box limits what gets through

# The policy and the gate

Lesson 5's `quark.py` plus 59 lines, mostly in `# — control flow —`. At the top of the section (abridged):

```
MAX_STEPS, MAX_TOKENS = 20, 200_000
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo", "du", ...}
DENY = re.compile(r"\bsudo\b|rm\s+-\w*[rf]|mkfs|git\s+push|(curl|wget).*\|\s*(ba)?sh|\s.env\b")
def guard(cmd):
    if DENY.search(cmd): return "blocked by policy"
    if not re.search(r"[;<>$`\n()]", cmd) and all(... in SAFE ...): return None
    answer = read(f"allow `{cmd}`? [y/N] ")
    return None if answer.lower() == "y" else "the person said no" + ...
```

## `guard()`: deny, allow, or ask

It returns `None` if the command may run, or the reason it may not:

1. Matches `DENY`: refused outright, and no one is asked
2. Only safe programs: it runs. No `;` `&` `<` `>` `$`, backtick, newline or `(`, and every piece between pipes starts with a `SAFE` program
3. Anything else goes to the person with Lesson 2's `read()`; a `y` lets it run
4. Any other answer is a no, and what they typed goes back to the model

# Limits, checked before every call

At the top of the loop, and where each call is counted (abridged):

```
if steps >= MAX_STEPS or spent >= MAX_TOKENS:
    print(f"[stopped: {steps} steps, {spent} tokens]")
    if not chat or (input := read("\n> ")) == "/q": break
    add(working_memory, {"role": "user", "content": input})
    steps, spent = 0, 0
    continue
...
with listening():
    response = call(max_tokens=16384, system=system(), tools=tools, messages=working_memory)
    output = response.content
    steps += 1
    spent += response.usage.input_tokens + response.usage.output_tokens + ...
```

In a one-shot run, a limit is the end. In a chat it hands back to you, and the counts start again after each new input.

# A refusal goes where the result would have

In the output loop, between printing the command and running it in the box (abridged):

```
print(f"$ {block.input['cmd']}")
if (no := guard(block.input["cmd"])):
    print(f"[{no}]")
    input.append({"type": "tool_result", ..., "content": no, "is_error": True})
    continue
```

- `continue` skips the run: the command never starts, and the model's next turn still makes sense
- A command that passes the gate runs in Lesson 5's box exactly as before

# The interrupt: listening for ESC

ESC is input from a person, so in `# — input —`, after `read()` (abridged):

```
ESC = threading.Event()                                # input: a person pressing ESC while quark works
SAYING = "[other self interrupted what you were saying – acknowledge]"
DOING = "[other self interrupted what you were doing – acknowledge]"
def watch(stop):
    ...
@contextlib.contextmanager
def listening():                                       # input: watch the keyboard only while quark thinks or acts
    if not sys.stdin.isatty(): yield; return
    ...
```

- `watch()` reads one key at a time in a thread; an ESC on its own sets `ESC`
- quark listens only while it thinks or acts, never while `read()` waits for you

# The interrupt: keep the work, tell the model

After an ESC (abridged):

```
if ESC.is_set():                # while it thought or said: keep what it said, run nothing
    ... "[your doing never reached the world]" ... SAYING ...
    ...
        doing = subprocess.Popen(..., stdout=subprocess.PIPE, ...)
        ...
            if ESC.is_set(): subprocess.run(..., "kill -9 -1", ...)
            ...
            if ESC.is_set(): done.stdout += "\n[your doing stopped before done]"
    ...
if ESC.is_set():                # while it acted: keep what it did
    add(working_memory, {"role": "user", "content": input + [..., DOING]})
```

Then the loop goes around: the model acknowledges it, and the run hands back to you.

# Safe commands run; anything else waits for you

`wc` is on the safe list, so nothing was asked:

```
$ wc -l notes.txt
3 notes.txt

notes.txt has 3 lines.
```

`rm` isn't, so the harness stopped at the prompt (I piped in `y` ; shortened):

```
$ rm a.log b.log; ls -A
allow `rm a.log b.log; ls -A`? [y/N] .env
.quark
notes.txt
```

# A no with a reason is an instruction

The answers piped in were `no, put it in a folder called drafts`, then `y` (shortened):

```
$ touch notes2.txt && ls -l notes2.txt  
allow `touch notes2.txt && ls -l notes2.txt`? [y/N] [the person said no: no, put it in a folder called drafts]  
$ mkdir -p drafts && touch drafts/notes2.txt && ls -l drafts/notes2.txt  
allow `mkdir -p drafts && touch drafts/notes2.txt && ls -l drafts/notes2.txt`? [y/N] -rw-rw-rw- 1 root root 0 Oct  6 19:15 drafts/notes2.txt
```

The model took the reason as an instruction and asked again.

## The deny list asks no one

```
$ cat .env  
[blocked by policy]
```

I managed to do nothing from your request. Policy blocked `cat .env`, and I didn't try `rm -rf .` because the same policy forbids it. I also didn't try to get around either block.

The model expected it, because it can read its own harness. The folder, `.env` included, is all still there.

# A limit stops the run before the next call

`MAX_STEPS` lowered to 2 in a copy, asking for three commands, one at a time:

```
$ date
Tue Oct 6 19:15:44 UTC 2026

$ pwd
/tmp/g6

[stopped: 2 steps, 14843 tokens]
```

`MAX_TOKENS` works the same way, but at 200,000 you'll rarely reach it.

# ESC stops the command, and keeps what it printed

In a terminal, in chat: count to ten a second at a time, `y`, then ESC four and a half seconds later (shortened):

```
$ for i in 1 2 3 4 5 6 7 8 9 10; do echo $i; sleep 1; done
1
2
3
4
5

[your doing stopped before done]
```

You interrupted the count, so it stopped after 5 and never reached 10. Do you want me to run it again from the start, or leave it there?

## Going further: what else guardrails can be

- **How a command is judged:** text patterns, a shell parser, per-tool rules, or a second model call
- **What the answers are:** allow once, for the session, always, or only with these arguments
- **Who gets asked:** the terminal, someone on Slack or Telegram, a second agent, or nobody
- **What counts as too much:** steps, tokens, dollars, wall-clock time, or repeating
- **How a run is interrupted,** and what the model is told about a refusal

## It can be a product on its own

- Policy services like **Open Policy Agent** decide *allowed or not* for a request, as data and not code
- Libraries like **NeMo Guardrails** and **Guardrails AI** wrap checks around what goes into and comes out of a model

If you're using one for what a tool may run, it's this layer.

## `guardrails.py`: finer answers, more limits

Built on Lesson 3's `control_flow.py`, with no system prompt:

- **Judged per piece:** `cd /tmp && rm x` is judged by `cd` *and* `rm`
- **Always:** `a` at the prompt adds those programs to `ALLOWED` for the rest of the run
- **A spending limit** in dollars, from each response's `usage`
- **A repeat limit:** the same command `MAX_REPEATS` times stops the run

## Two questions covered five commands

I answered `a` to `mkdir` and to the first `touch` :

```
$ mkdir /tmp/gdemo
allow `mkdir /tmp/gdemo`? (mkdir isn't on the allowed list) [y]es, [a]lways, or say why not:
$ cat .env
[blocked by policy: secrets files are off limits]
$ touch /tmp/gdemo/a.txt
allow `touch /tmp/gdemo/a.txt`? (touch isn't on the allowed list) [y]es, [a]lways, or say why not:
$ touch /tmp/gdemo/b.txt

$ ls /tmp/gdemo
a.txt
b.txt
```

# The rule: decide before the harness does it

**The rule:** check each tool request where control flow turns it into a command, and the run where it goes around again. Say no in a way the model can read, and give the person a way in.

What guardrails never do, from inside control flow:

- **Model interface:** sends and receives as before; guardrails read `usage` and the tool request
- **Output:** an allowed command runs in the box exactly as before
- **Context:** a refusal is an ordinary tool result in the result's slot
- **Input:** Lesson 2's `read()`, plus a watcher that hears ESC while quark works

## What's missing: you can stop a run, not see one

- How long did each step take?
- How many tokens, and what did that cost?
- Which command did the guard refuse, and which did the box kill?
- The episode has every message, but it's written for the model: no timings, no token counts, no exit codes

# **Next: Observability**

Lesson 7